

Guía de Actividad

Desarrollo de un Sistema Cognitivo usando MCP y Bases de Datos

Curso: Computación Cognitiva para Big Data

Tema: Conexión a herramientas externas mediante Model Context Protocol

Presentación

Las organizaciones modernas requieren sistemas capaces de consultar, procesar y analizar información almacenada en bases de datos, servicios web y plataformas externas. En este contexto, los sistemas cognitivos apoyados en modelos de lenguaje pueden extender sus capacidades mediante herramientas que les permitan interactuar con datos reales.

El *Model Context Protocol* (MCP) permite conectar modelos de lenguaje con herramientas externas, tales como bases de datos, archivos, APIs y servicios computacionales. En esta actividad se propone el desarrollo de un servidor MCP que permita gestionar información de un inventario mediante operaciones CRUD y consultas analíticas básicas.

Objetivo de la actividad

Diseñar e implementar un servidor MCP en Python que permita realizar operaciones CRUD y consultas analíticas sobre una base de datos de inventario, con el propósito de comprender cómo un sistema cognitivo puede interactuar con herramientas externas.

4.1 Saberes cognitivos

- Sistemas cognitivos.
- Model Context Protocol.
- Bases de datos relacionales.
- Operaciones CRUD.
- Herramientas externas para modelos de lenguaje.

4.2 Saberes procedimentales

- Crear una base de datos SQLite.
- Implementar funciones en Python.
- Crear herramientas MCP mediante FastMCP.

- Ejecutar pruebas funcionales del servidor.
- Analizar información almacenada en la base de datos.

4.3 Saberes actitudinales

- Trabajo colaborativo.
- Responsabilidad en la entrega de evidencias.
- Buenas prácticas de programación.
- Claridad en la documentación técnica.

Requisitos técnicos

Para desarrollar la actividad se requiere:

- Python 3.8 o superior.
- SQLite3.
- Librería `fastmcp`.
- Editor de código como Visual Studio Code.
- Git y GitHub para la entrega del repositorio.

Instalación de la librería principal:

```
1 pip install fastmcp
```

Estructura sugerida del proyecto

```
1 mcp_inventory/  
2 |-- server.py  
3 |-- database.py  
4 |-- requirements.txt  
5 |-- README.md  
6 |-- inventory.db
```

Desarrollo de la actividad

Parte 1. Creación de la base de datos

Cree un archivo llamado `database.py` con el siguiente código:

```
1 import sqlite3
2
3 DB_NAME = "inventory.db"
4
5 def init_db():
6     conn = sqlite3.connect(DB_NAME)
7     cursor = conn.cursor()
8
9     cursor.execute("""
10     CREATE TABLE IF NOT EXISTS productos (
11         id INTEGER PRIMARY KEY AUTOINCREMENT,
12         nombre TEXT NOT NULL,
13         categoria TEXT,
14         cantidad INTEGER NOT NULL,
15         precio REAL NOT NULL
16     )
17     """)
18
19     conn.commit()
20     conn.close()
```

Parte 2. Creación del servidor MCP

Cree un archivo llamado `server.py`.

```
1 from mcp.server.fastmcp import FastMCP
2 import sqlite3
3 from database import init_db, DB_NAME
4
5 init_db()
6
7 mcp = FastMCP("InventarioDB")
8
9 def get_connection():
10     return sqlite3.connect(DB_NAME)
11
12 @mcp.tool()
13 def crear_producto(nombre: str, categoria: str, cantidad: int, precio
14     : float) -> str:
15     conn = get_connection()
16     cursor = conn.cursor()
```

```
17     cursor.execute(  
18         "INSERT INTO productos (nombre, categoria, cantidad, precio)  
19         VALUES (?, ?, ?, ?)",  
20         (nombre, categoria, cantidad, precio)  
21     )  
22     conn.commit()  
23     conn.close()  
24  
25     return "Producto creado exitosamente"  
26  
27 @mcp.tool()  
28 def consultar_producto(id: int) -> dict:  
29     conn = get_connection()  
30     cursor = conn.cursor()  
31  
32     cursor.execute("SELECT * FROM productos WHERE id = ?", (id,))  
33     row = cursor.fetchone()  
34  
35     conn.close()  
36  
37     if row:  
38         return {  
39             "id": row[0],  
40             "nombre": row[1],  
41             "categoria": row[2],  
42             "cantidad": row[3],  
43             "precio": row[4]  
44         }  
45  
46     return {"error": "Producto no encontrado"}  
47  
48 @mcp.tool()  
49 def actualizar_producto(id: int, cantidad: int) -> str:  
50     conn = get_connection()  
51     cursor = conn.cursor()  
52  
53     cursor.execute(  
54         "UPDATE productos SET cantidad = ? WHERE id = ?",  
55         (cantidad, id)  
56     )  
57  
58     conn.commit()  
59     conn.close()  
60  
61     return "Producto actualizado correctamente"  
62  
63 @mcp.tool()
```

```
64 def eliminar_producto(id: int) -> str:
65     conn = get_connection()
66     cursor = conn.cursor()
67
68     cursor.execute("DELETE FROM productos WHERE id = ?", (id,))
69
70     conn.commit()
71     conn.close()
72
73     return "Producto eliminado correctamente"
74
75 @mcp.tool()
76 def listar_productos() -> list:
77     conn = get_connection()
78     cursor = conn.cursor()
79
80     cursor.execute("SELECT * FROM productos")
81     rows = cursor.fetchall()
82
83     conn.close()
84
85     return [
86         {
87             "id": row[0],
88             "nombre": row[1],
89             "categoria": row[2],
90             "cantidad": row[3],
91             "precio": row[4]
92         }
93         for row in rows
94     ]
```

Parte 3. Herramientas cognitivas y analíticas

Agregue al archivo `server.py` las siguientes herramientas:

```
1 @mcp.tool()
2 def calcular_valor_total_inventario() -> dict:
3     conn = get_connection()
4     cursor = conn.cursor()
5
6     cursor.execute("SELECT SUM(cantidad * precio) FROM productos")
7     total = cursor.fetchone()[0]
8
9     conn.close()
10
11     return {"valor_total_inventario": total if total else 0}
12
```

```
13 @mcp.tool()
14 def productos_agotados() -> list:
15     conn = get_connection()
16     cursor = conn.cursor()
17
18     cursor.execute("SELECT * FROM productos WHERE cantidad = 0")
19     rows = cursor.fetchall()
20
21     conn.close()
22
23     return [
24         {
25             "id": row[0],
26             "nombre": row[1],
27             "categoria": row[2],
28             "cantidad": row[3],
29             "precio": row[4]
30         }
31         for row in rows
32     ]
33
34 @mcp.tool()
35 def producto_mas_costoso() -> dict:
36     conn = get_connection()
37     cursor = conn.cursor()
38
39     cursor.execute("SELECT * FROM productos ORDER BY precio DESC
40                     LIMIT 1")
41     row = cursor.fetchone()
42
43     conn.close()
44
45     if row:
46         return {
47             "id": row[0],
48             "nombre": row[1],
49             "categoria": row[2],
50             "cantidad": row[3],
51             "precio": row[4]
52         }
53
54     return {"error": "No hay productos registrados"}
55
56 @mcp.tool()
57 def estadisticas_inventario() -> dict:
58     conn = get_connection()
59     cursor = conn.cursor()
```

```
60     cursor.execute("SELECT COUNT(*), AVG(cantidad), AVG(precio), SUM(  
61         cantidad * precio) FROM productos")  
62     total_productos, promedio_cantidad, promedio_precio, valor_total  
63     = cursor.fetchone()  
64  
65     conn.close()  
66  
67     return {  
68         "total_productos": total_productos,  
69         "promedio_cantidad": promedio_cantidad,  
70         "promedio_precio": promedio_precio,  
71         "valor_total": valor_total if valor_total else 0  
72     }
```

Finalmente, al cierre del archivo `server.py`, agregue:

```
1 if __name__ == "__main__":  
2     mcp.serve()
```

Archivo requirements.txt

Cree un archivo llamado `requirements.txt` con el siguiente contenido:

```
1 fastmcp
```

Pruebas sugeridas

El estudiante debe probar las siguientes acciones:

1. Crear mínimo cinco productos.
2. Consultar un producto por su identificador.
3. Actualizar la cantidad de un producto.
4. Eliminar un producto.
5. Listar todos los productos registrados.
6. Calcular el valor total del inventario.
7. Consultar productos agotados.
8. Identificar el producto más costoso.
9. Consultar estadísticas generales del inventario.

Preguntas de análisis

Responda en el informe:

1. ¿Qué papel cumple MCP en la construcción de sistemas cognitivos?
2. ¿Por qué es importante conectar un modelo de lenguaje con herramientas externas?
3. ¿Qué ventajas ofrece el uso de una base de datos frente al manejo de archivos planos?
4. ¿Cómo podría ampliarse esta solución para un escenario de Big Data?
5. ¿Qué riesgos se deben considerar al permitir que un modelo de IA consulte o modifique datos?

Entregables

El estudiante debe entregar:

- Enlace al repositorio de GitHub.
- Código fuente completo.
- Archivo `README.md` con instrucciones de instalación y ejecución.
- Capturas de pantalla de las pruebas realizadas.
- Informe técnico en PDF.